

Quantized Guardians: Accuracy-Neutral Decision Churn in Compressed Prompt-Injection Detectors

Muneeb Anjum
Independent Researcher
ORCID: 0009-0001-2042-7656

Abstract—Lightweight transformer classifiers are the first line of defense against prompt injection and jailbreak attacks in deployed large language model (LLM) applications, and they are routinely compressed to low-precision formats to cut latency and memory on CPU gateways. Practitioners typically validate a compressed guardrail by checking that aggregate metrics such as F1 or AUROC are preserved. We show that this check is insufficient. We evaluate four open-source detectors (ProtectAI DeBERTa-v3-v2, PIGuard, Deepset DeBERTa-v3, and FMOps DistilBERT) under FP32, BF16, simulated W8A8 dynamic INT8, and 4-bit round-to-nearest (RTN) weight quantization with group-wise and per-channel scales (and, in an extended study, native INT8 and GPTQ/AWQ-style 4-bit)—on 2,755 inputs drawn from four public benchmarks plus 900 character-perturbed attacks. Aggregate F1 changes by at most 2.1 points, yet quantization silently rewrites between 0.36% and 5.7% of individual decisions. For the strongest detector, 4-bit per-channel quantization flips 108 decisions—61 inputs become wrong and 47 become right—so accuracy is statistically unchanged (exact McNemar $p = 0.21$) while 51 attacks the full-precision model blocked now pass, and homoglyph-attack recall falls from 96.0% to 88.7%. The direction of drift is detector- and format-specific: 8-bit quantization mainly adds false alarms (up to 52 on one detector), while 4-bit per-channel quantization mainly opens new misses (51 and 66 attacks on the two strongest detectors). Threshold re-calibration on held-out data repairs threshold-shift drift but not per-channel 4-bit churn. In the extended study, the default native per-tensor dynamic INT8 paths of PyTorch and ONNX Runtime flip 30.05% and 38.19% of ProtectAI-v2 decisions and 31.54% of PIGuard decisions (PIGuard then flags no attack), which a per-token simulation misses (1.78% and 3.05%); the collapse reproduces on x86_64 and aarch64 CPUs in aggregate but not input by input; and GPTQ did not reduce churn relative to RTN for three of four detectors. We argue that compressed guardrails must be validated with *instance-level agreement* and adversarial slices, not aggregate scores, and we describe a practical evaluation protocol for doing so.

Index Terms—LLM security, prompt injection, jailbreak detection, guardrails, model quantization, robustness evaluation

I. INTRODUCTION

Prompt injection—adversarial text that overrides an LLM application’s intended instructions—is now a central security concern for LLM-integrated systems [1]–[3]. A widely adopted mitigation is an input *guardrail*: a small encoder classifier, usually a fine-tuned DeBERTa-v3 [4] or DistilBERT [5], that scores every prompt or retrieved document before it reaches the LLM. Examples include Meta’s Prompt Guard [6], ProtectAI’s DeBERTa detector [7], and PIGuard [8].

Because a guardrail sits on the critical path of every request, operators compress it. INT8 ONNX exports of popular

detectors are published on model hubs, and low-bit weight quantization from the LLM literature [9]–[11] is increasingly applied to encoders. The common acceptance test is simple: re-run a benchmark and confirm that F1 or AUROC is roughly unchanged.

A security filter, however, is not judged by its average. An attacker needs only one input that passes; an operator’s alerting, allow-lists, and thresholds are tuned to the *specific* behavior of the model they validated. If quantization preserves the average while reshuffling which inputs pass, the aggregate check gives false assurance. Prior work has studied evasion of full-precision guardrails [12], [13], over-defense [8], and long-context blind spots [14], and separate work has studied how quantization affects the trustworthiness of generative LLMs [15], [16]. To our knowledge, the effect of quantization on the *detectors themselves* has not been measured systematically.

Contributions. (1) We define an instance-level evaluation protocol for compressed guardrails: decision flip rate, missed-attack and new-false-positive counts relative to the full-precision model, paired McNemar tests, and adversarial-slice recall. (2) We apply it to four public detectors under up to five numeric formats on 2,755 inputs and show that aggregate metrics are near-invariant while up to 5.7% of decisions change. (3) We show the drift is detector- and format-specific in direction, so no single post-quantization threshold rule is safe. (4) We test held-out threshold re-calibration as a mitigation and show it fixes threshold-shift drift but not accuracy-neutral churn. (5) We run native INT8 kernels and GPTQ/AWQ-style 4-bit quantization and compare them with simulation, across two CPU architectures.

II. BACKGROUND AND RELATED WORK

Prompt-injection detection. Perez and Ribeiro [1] introduced goal-hijacking prompts; Greshake et al. [2] extended the threat to indirect injection through retrieved content; Liu et al. [3] formalized attacks and defenses. Classifier guardrails are the most deployed defense. Li et al. [8] identified over-defense—benign prompts containing trigger words being flagged—and released the NotInject benchmark and PIGuard. Hackett et al. [12] showed character-level perturbations (homoglyphs, zero-width characters) evade several commercial and open guardrails, and Rahman and Alouani [13] built universal evasive suffixes with gradient search. Recent work identified that 512-token guardrail windows can be exploited

by spreading an injection across segments [14]. All of these study full-precision detectors.

Quantization and trust. Post-training quantization (PTQ) maps weights, and optionally activations, to low-bit integers [17]. LLM.int8() [9], GPTQ [10], and AWQ [11] preserve perplexity at 8 and 4 bits. Hong et al. [15] showed compression can degrade trustworthiness dimensions of LLMs that perplexity does not reveal, and Egashira et al. [16] showed quantization can be weaponized to unlock hidden malicious behavior. Our work brings this lens to the small classifiers that guard LLMs, where the relevant failure is a changed security decision rather than changed generated text.

III. METHODOLOGY

A. Detectors

We study four publicly available binary detectors from Hugging Face: *ProtectAI-v2* (protectai/deberta-v3-base-prompt-injection-v2) [7]; *PIGuard* (leolee99/PIGuard, DeBERTa-v3-base) [8]; *Deepset* (deepset/deberta-v3-base-injection); and *FMops* (fmops/distilbert-prompt-injection). Meta’s Prompt Guard models are gated and were excluded. We use each model’s attack-class softmax probability p and the default decision threshold $\tau = 0.5$, inputs truncated to 512 tokens.

B. Numeric formats

Starting from the released FP32 checkpoint we construct: **BF16**, casting all weights and activations to bfloat16; **W8A8**, simulated dynamic INT8 with symmetric per-output-channel weight scales and symmetric per-token activation scales applied at the input of every linear layer, mirroring dynamic INT8 kernels used in ONNX Runtime and PyTorch; **W4-g128**, 4-bit asymmetric round-to-nearest (RTN) weight-only quantization with groups of 128 input channels; and **W4-pc**, 4-bit RTN with a single scale per output channel. All linear layers, including attention projections, feed-forward layers, and the classification head, are quantized; embeddings are kept in full precision. All formats are evaluated for ProtectAI-v2, Deepset, and FMops; BF16 is evaluated on ProtectAI-v2 only, as a near-lossless reference, and compute limits restricted PIGuard to FP32 and W4-pc. W8A8 and W4 formats are simulated by quantize–dequantize (“fake quantization”), which reproduces the numerical rounding error of integer kernels without depending on hardware-specific kernels.

C. Evaluation data

We assemble 2,755 inputs from four public sources: the Deepset prompt-injection test split (116 inputs); a class-balanced sample of 1,000 inputs from the Safe-Guard prompt-injection test split [12]; the Jailbreak-Classification test split (400 inputs, 141 jailbreaks); and all 339 benign NotInject prompts [8], which contain injection trigger words and measure over-defense. Following Hackett et al. [12], we also create three adversarial slices from 300 randomly selected Safe-Guard attacks: *homoglyph* (each of a, e, o, p, c, i, x, y replaced with its Cyrillic look-alike with probability 0.5), *zero-width* (U+200B inserted after letters with probability 0.3),

and *leet* (a, e, i, o, s, t mapped to digits with probability 0.5). Seeds are fixed. Some detectors may have seen parts of these benchmarks in training; this does not affect our main quantity of interest, which is the change relative to the same model’s FP32 decisions.

D. Metrics

On the clean pool (Deepset, Safe-Guard, Jailbreak-CLs; $n=1,516$) we report AUROC, F1, true- and false-positive rates (TPR, FPR), and 10-bin expected calibration error (ECE) [18]. Over-defense is FPR on NotInject; robustness is detection rate on each adversarial slice. Our instance-level metrics, computed over all 2,755 inputs relative to the FP32 model, are: the *flip rate* $\frac{1}{n} \sum_i \mathbb{1}[\hat{y}_i^q \neq \hat{y}_i^{\text{fp32}}]$ with a 1,000-sample bootstrap 95% CI; *missed attacks*, true attacks flagged by FP32 but passed by the quantized model; *new false positives*, benign inputs newly flagged; and the maximum absolute probability change $\max_i |p_i^q - p_i^{\text{fp32}}|$. We test whether correctness changes with an exact McNemar test on paired outcomes.

E. Mitigation: threshold re-calibration

A natural repair is to re-tune τ for the quantized model. We split inputs 50/50 at random, choose τ^* on the first half to maximize agreement with FP32 decisions, and report the flip rate on the second half at $\tau = 0.5$ and at τ^* .

IV. RESULTS

A. Aggregate metrics hide change

Table I reports aggregate metrics. For ProtectAI-v2, the strongest detector, AUROC stays within 0.977–0.980 and F1 within 0.888–0.903 across all formats; a practitioner applying the usual acceptance test would ship any of them. Two of the four public detectors (Deepset and FMops) flag most benign inputs at FP32 on this mixed distribution (FPR > 0.77), consistent with the over-defense reported by [8]. PIGuard has the lowest over-defense (NotInject FPR 0.115) but, notably, detects only 18.0% of homoglyph attacks and 0.3% of zero-width attacks even at full precision.

B. Decisions churn underneath

Table II and Fig. 2 show that every low-bit format changes a material fraction of decisions. For ProtectAI-v2, W4-pc flips 3.92% of inputs (95% CI 3.19–4.61%): 51 attacks that the FP32 model blocked now pass, and 10 benign inputs are newly blocked. Measured against ground truth, 61 inputs became wrong and 47 became right, so overall correctness is statistically unchanged (McNemar $p = 0.21$). This is *accuracy-neutral churn*: the model is, on average, as good—but it is a different filter, and its new blind spots are exactly the inputs the operator never saw fail. Individual probabilities move by up to 0.99, i.e., confident “attack” scores become confident “benign” scores (Fig. 1). BF16 is the only format that is effectively lossless (0.15% flips, McNemar $p = 0.63$).

TABLE I

AGGREGATE DETECTION METRICS ON THE CLEAN POOL (DEEPSET, SAFE-GUARD, JAILBREAK-CLS; $n = 1,516$) AND OVER-DEFENSE ON NOTINJECT ($n = 339$ BENIGN). THRESHOLD 0.5. AGGREGATE NUMBERS BARELY MOVE UNDER QUANTIZATION.

Detector	Format	AUROC	F1	TPR	FPR	NotInject FPR	ECE
ProtectAI-v2	FP32	0.977	0.889	0.807	0.007	0.434	0.094
	BF16	0.977	0.888	0.806	0.007	0.437	0.094
	W8A8	0.978	0.903	0.833	0.010	0.460	0.080
	W4-g128	0.976	0.894	0.816	0.007	0.442	0.088
	W4-pc	0.980	0.895	0.817	0.009	0.401	0.086
PIGuard	FP32	0.931	0.807	0.705	0.036	0.115	0.135
	W4-pc	0.958	0.787	0.676	0.037	0.133	0.124
Deepset	FP32	0.710	0.687	0.994	0.775	0.714	0.415
	W8A8	0.692	0.681	0.997	0.801	0.805	0.430
	W4-g128	0.627	0.683	0.997	0.793	0.776	0.424
	W4-pc	0.741	0.686	0.986	0.763	0.729	0.387
FMOps	FP32	0.611	0.672	0.973	0.795	0.717	0.435
	W8A8	0.612	0.671	0.969	0.791	0.708	0.437
	W4-g128	0.618	0.673	0.981	0.804	0.735	0.437
	W4-pc	0.625	0.673	0.977	0.798	0.720	0.434

TABLE II

INSTANCE-LEVEL DECISION CHANGES RELATIVE TO THE FP32 DETECTOR OVER ALL 2,755 INPUTS. *Missed*: ATTACKS THE FP32 MODEL FLAGGED THAT THE QUANTIZED MODEL PASSES. *New FP*: BENIGN INPUTS NEWLY FLAGGED.

Det.	Fmt	Flip %	Missed	New FP	$\max \Delta p $
ProtectAI-v2	BF16	0.15	2	1	0.427
	W8A8	1.78	0	11	0.933
	W4-g128	1.52	11	7	0.985
	W4-pc	3.92	51	10	0.993
PIGuard	W4-pc	5.74	66	24	0.987
Deepset	W8A8	2.00	0	52	0.977
	W4-g128	1.34	0	35	0.957
	W4-pc	2.00	6	22	0.934
FMOps	W8A8	0.36	4	0	0.366
	W4-g128	0.76	0	13	0.811
	W4-pc	0.80	0	11	0.813

TABLE III

DETECTION RATE (%) ON 300 CHARACTER-PERTURBED ATTACKS PER TRANSFORM.

Det.	Fmt	Homoglyph	Zero-width	Leet
ProtectAI-v2	FP32	96.0	100.0	91.7
	BF16	96.3	100.0	91.3
	W8A8	98.7	100.0	95.7
	W4-g128	95.0	100.0	93.7
	W4-pc	88.7	100.0	88.7
PIGuard	FP32	18.0	0.3	84.0
	W4-pc	18.3	0.0	85.7
Deepset	FP32	99.7	100.0	100.0
	W8A8	100.0	100.0	100.0
	W4-g128	99.7	100.0	100.0
	W4-pc	99.7	100.0	100.0
FMOps	FP32	100.0	98.7	100.0
	W8A8	100.0	98.3	100.0
	W4-g128	100.0	99.3	100.0
	W4-pc	100.0	98.7	100.0

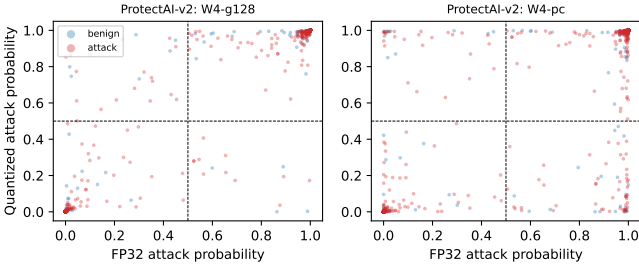


Fig. 1. Per-input attack probability before (x) and after (y) 4-bit quantization of ProtectAI-v2. Points off the diagonal crossing the dashed 0.5 lines are flipped decisions; red points in the lower-right quadrant are attacks that now pass.

C. Drift direction is detector-specific

The same recipe pushes different detectors in different directions. Under W8A8, ProtectAI-v2 gains 11 new false positives and loses no attacks, while Deepset gains 52 new

false positives (NotInject FPR rises from 0.714 to 0.805); both shifts are significant (McNemar $p < 10^{-3}$). Under W4-g128, Deepset’s AUROC drops from 0.710 to 0.627—an 8-point loss in ranking quality—while its F1 is unchanged at 0.683, because its decisions at $\tau=0.5$ were already saturated. FMops, the only DistilBERT model, is the most numerically stable (at most 0.80% flips), but its FP32 decisions are already close to “flag everything” (FPR 0.80), leaving little room to drift. PIGuard under W4-pc shows the largest churn in our study: 5.74% of decisions flip, 66 previously blocked attacks pass and 24 benign inputs are newly blocked, while its AUROC actually *rises* from 0.931 to 0.958 and F1 falls only from 0.807 to 0.787.

D. Adversarial slices amplify the effect

Table III shows that drift is concentrated where models are least certain. For ProtectAI-v2, clean TPR changes by

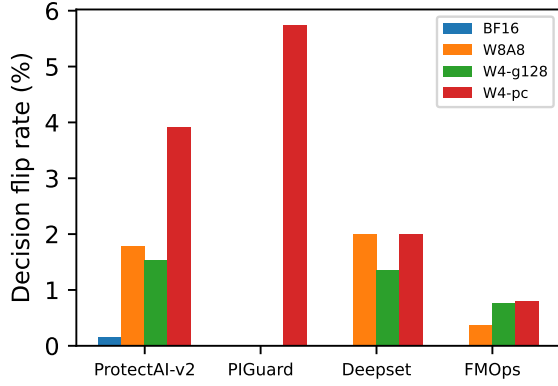


Fig. 2. Decision flip rate relative to FP32 by detector and numeric format.

TABLE IV
HELD-OUT FLIP RATE (%) RELATIVE TO FP32 BEFORE AND AFTER THRESHOLD RE-CALIBRATION, WITH McNemar p FOR CHANGE IN CORRECTNESS (ALL INPUTS).

Det.	Fmt	τ^*	@0.5	@ τ^*	McNemar p
ProtectAI-v2	BF16	0.46	0.00	0.07	0.62
	W8A8	0.89	2.00	1.11	<0.001
	W4-g128	0.75	1.63	1.04	0.44
	W4-pc	0.03	4.00	4.44	0.21
PIGuard	W4-pc	0.61	5.18	5.55	0.09
Deepset	W8A8	0.95	2.15	1.41	<0.001
	W4-g128	0.95	1.55	0.52	<0.001
	W4-pc	0.34	2.22	2.00	1.00
FMops	W8A8	0.36	0.44	0.37	0.75
	W4-g128	0.72	0.96	0.67	0.38
	W4-pc	0.53	0.96	0.96	1.00

under one point under W4-pc, but homoglyph recall falls from 96.0% to 88.7% and leet recall from 91.7% to 88.7%. Character-perturbed attacks lie close to the decision boundary, so small numeric perturbations push them across it. Since these are precisely the inputs an adaptive attacker produces [12], the security cost of quantization is larger than the clean-benchmark cost suggests.

E. Does re-calibration help?

Table IV reports held-out flip rates. Re-tuning τ roughly halves churn when the dominant effect is a global score shift: ProtectAI-v2 W8A8 drops from 2.00% to 1.11% and Deepset W4-g128 from 1.55% to 0.52%. It does *not* help W4-pc on ProtectAI-v2 (4.00% to 4.44%) or PIGuard (5.18% to 5.55%), where flips are input-specific rather than a global shift. The chosen thresholds also vary widely (from 0.03 to 0.95), so a threshold tuned for the FP32 model should never be carried over to a quantized one.

F. Extended study: native INT8, engines, and GPTQ

Native versus simulated INT8. The default one-line dynamic INT8 paths are far more damaging than the simulations above (Table V). PyTorch dynamic INT8 flips 30.05% of

TABLE V
NATIVE INTEGER EXECUTION VERSUS SIMULATED INT8. FLIP RATES ARE MEASURED AGAINST EACH PATH’S OWN FP32 REFERENCE (PYTORCH FP32 FOR INT8 PYTORCH; ONNX RUNTIME FP32 FOR INT8 ONNX RT). J : JACCARD OVERLAP OF THE TWO FLIP SETS; AGREE: FRACTION OF IDENTICAL FINAL DECISIONS BETWEEN THE NATIVE AND SIMULATED MODELS.

Detector	Native	Simulated	Flip _{nat} %	Flip _{sim} %	J	Agree
ProtectAI-v2	INT8 PyTorch	W8A8 ^s (token)	30.05	30.05	1.00	1.00
	INT8 PyTorch	W8A8 ^s (tensor)	30.05	30.05	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (token)	38.19	38.19	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (tensor)	38.19	38.19	1.00	1.00
	INT8 PyTorch (per-ch.)	W8A8 ^s (token)	13.97	13.97	1.00	1.00
	INT8 PyTorch (per-ch.)	W8A8 ^s (tensor)	13.97	13.97	1.00	1.00
	INT8 ONNX RT (per-ch.)	W8A8 ^s (token)	13.50	13.50	1.00	1.00
	INT8 ONNX RT (per-ch.)	W8A8 ^s (tensor)	13.50	13.50	1.00	1.00
PIGuard	INT8 PyTorch	W8A8 ^s (token)	31.54	31.54	1.00	1.00
	INT8 PyTorch	W8A8 ^s (tensor)	31.54	31.54	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (token)	31.54	31.54	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (tensor)	31.54	31.54	1.00	1.00
Deepset	INT8 PyTorch	W8A8 ^s (token)	10.13	10.13	1.00	1.00
	INT8 PyTorch	W8A8 ^s (tensor)	10.13	10.13	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (token)	10.34	10.34	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (tensor)	10.34	10.34	1.00	1.00
FMops	INT8 PyTorch	W8A8 ^s (token)	1.23	1.23	1.00	1.00
	INT8 PyTorch	W8A8 ^s (tensor)	1.23	1.23	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (token)	0.54	0.54	1.00	1.00
	INT8 ONNX RT	W8A8 ^s (tensor)	0.54	0.54	1.00	1.00

ProtectAI-v2 decisions (514 missed attacks, 141 new false positives; AUROC 0.977→0.542) and ONNX Runtime INT8 flips 38.19% (805 missed). For PIGuard both native paths flip 31.54% of decisions, miss all 801 attacks FP32 blocked, and flag nothing (F1 0.000). Deepset flips about 10% (265 and 280 new false positives, no missed attacks); FMops flips 1.23% and 0.54%. A per-token simulation reports only 1.78%, 3.05%, 2.00% and 0.36%, whereas a per-tensor simulation matches the native aggregate effect. In a 150-input ablation on ProtectAI-v2, per-tensor weights alone flip 36.7% of decisions and per-tensor activations alone 21.3%, so each alone is sufficient to cause the collapse; per-channel weights with per-token activations flip 1.3%.

Engine comparison. We repeated PyTorch dynamic INT8 and FP32 runs for ProtectAI-v2 and FMops on GitHub Actions x86_64 CPU runners, versus the original 2-vCPU aarch64 CPU. FP32 decisions were identical on both. Native INT8 flip rates agree within 0.4 points (ProtectAI-v2 29.73% vs 30.05%; FMops 1.34% vs 1.23%), but 117 of 2,755 ProtectAI-v2 decisions and 7 FMops decisions differ between platforms.

Calibration-based 4-bit. GPTQ (three calibration seeds) did not reduce churn relative to W4 per-channel RTN for three of four detectors (Table VI): ProtectAI-v2 $9.39 \pm 0.67\%$ vs 3.92% , PIGuard $15.69 \pm 0.95\%$ vs 5.74% , Deepset $3.91 \pm 1.29\%$ vs 2.00% ; FMops $0.64 \pm 0.23\%$ vs 0.80% . Flip sets differ across seeds (mean pairwise Jaccard 0.29–0.76). A single-seed AWQ-style variant flipped 3.01% (ProtectAI-v2) and 0.69% (FMops).

TABLE VI

CALIBRATION-BASED 4-BIT QUANTIZATION VERSUS RTN (W4 PER-CHANNEL UNLESS NOTED). GPTQ/AWQ VALUES ARE MEAN $\pm$ SAMPLE STD OVER CALIBRATION SEEDS (128 CALIBRATION PROMPTS PER SEED DRAWN FROM TRAINING SPLITS DISJOINT FROM THE EVALUATION SET). J_{seed} : MEAN PAIRWISE JACCARD OVERLAP OF FLIP SETS ACROSS SEEDS; J_{RTN} : MEAN OVERLAP WITH THE RTN FLIP SET.

Detector	Method	Seeds	Flip %	Missed	New FP	F1	J_{seed}	J_{RTN}
ProtectAI-v2	RTN	–	3.92	51	10	0.895	–	–
	AWQ	1	3.01	18.0	10.0	0.907	–	0.35
	GPTQ (g128)	1	1.31	10.0	7.0	0.888	–	0.32
	GPTQ	3	9.39 $\pm$ 0.67	184.0 $\pm$ 14.80	1.0 $\pm$ 1.73	0.785	0.66	0.24
PIGuard	RTN	–	5.74	66	24	0.787	–	–
	GPTQ	3	15.69 $\pm$ 0.95	385.3 $\pm$ 23.76	0.0 $\pm$ 0.00	0.686	0.76	0.16
Deepset	RTN	–	2.00	6	22	0.686	–	–
	GPTQ	3	3.91 $\pm$ 1.29	43.7 $\pm$ 40.05	57.0 $\pm$ 7.00	0.680	0.29	0.15
FMOps	RTN	–	0.80	0	11	0.673	–	–
	AWQ	1	0.69	2.0	11.0	0.672	–	0.46
	GPTQ	3	0.64 $\pm$ 0.23	0.7 $\pm$ 1.15	12.3 $\pm$ 4.93	0.673	0.31	0.35

Compute. Early runs used a local 2-vCPU aarch64 CPU; GPTQ/AWQ, the PIGuard and Deepset grids and the x86_64 replicates ran on GitHub Actions x86_64 CPU runners. No GPU was used.

V. DISCUSSION

Recommendations. (i) Validate a compressed guardrail by *agreement with its full-precision parent*, reporting flip rate and missed-attack count, not only F1/AUROC. (ii) Include adversarial slices (homoglyph, zero-width, leet, and others) in the acceptance suite, since drift concentrates there. (iii) Re-calibrate thresholds per deployed artifact. (iv) Prefer BF16 or group-wise 4-bit over per-channel 4-bit for guardrails; per-channel 4-bit produced the largest, least repairable churn. (v) Treat each quantized artifact as a new model for red-teaming purposes.

Limitations. INT8 and INT4 are simulated with quantize–dequantize rather than executed with integer kernels; real kernels add accumulation and operator-fusion differences that may increase or decrease churn. We evaluate RTN only; calibration-based methods such as GPTQ or AWQ may churn less. Experiments were run on CPU with four detectors, a partial format grid for PIGuard, and one seed per perturbation, and some benchmarks may overlap detectors’ training data. Our adversarial slices use non-adaptive perturbations; an attacker who searches against the quantized artifact specifically would likely find more failures.

VI. CONCLUSION

Quantizing a prompt-injection detector can leave its benchmark score intact while changing which attacks it lets through. Across four public detectors and up to five numeric formats we observe up to 5.7% of decisions rewritten with near-identical aggregate metrics, detector-specific drift directions, amplified losses on character-perturbed attacks, and churn that threshold re-calibration cannot fully repair. Default native per-tensor INT8 collapsed two of the four detectors (30–38% flips), simulation with finer granularity understated this, and GPTQ

was not a reliable remedy. Guardrail compression should be evaluated at the level of individual decisions on the exported artifact.

REPRODUCIBILITY

All detectors and datasets are public. Data construction, quantization code, and per-input scores are available at <https://github.com/horizonbymuneeb/quantized-guardians>.

ACKNOWLEDGMENT

AI tools were used to assist with running experiments and drafting this manuscript. The author reviewed all content and takes responsibility for it. This work received no external funding, and the author declares no conflicts of interest.

REFERENCES

- [1] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” in *NeurIPS ML Safety Workshop*, 2022.
- [2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. ACM AISeC*, 2023.
- [3] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. USENIX Security*, 2024.
- [4] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” in *Proc. ICLR*, 2023.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” arXiv:1910.01108, 2019.
- [6] A. Dubey *et al.*, “The Llama 3 herd of models,” arXiv:2407.21783, 2024.
- [7] ProtectAI.com, “Fine-tuned DeBERTa-v3-base for prompt injection detection (v2),” Hugging Face model card, 2024.
- [8] H. Li and X. Liu, “PIGuard: Prompt injection guardrail via mitigating overdefense for free,” in *Proc. ACL*, 2025 (earlier version: “InjecGuard,” arXiv:2410.22770).
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. NeurIPS*, 2022.
- [10] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023.
- [11] J. Lin *et al.*, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proc. MLSys*, 2024.

- [12] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems," in *Proc. 1st Workshop on LLM Security (LLMSEC)*, 2025, arXiv:2504.11168.
- [13] M. J. Rahman and I. Alouani, "Bypassing prompt injection detectors through evasive injections," arXiv:2602.00750, 2026.
- [14] Y. Zhou, C. Zhu, J. Wang, X. He, Y. Zhai, K. Sun, M. Wei, and J. Xiong, "Prompt overflow: What the guardrail inspects is not what the model infers," arXiv:2605.23196, 2026.
- [15] J. Hong *et al.*, "Decoding compressed trust: Scrutinizing the trustworthiness of efficient LLMs under compression," in *Proc. ICML*, 2024.
- [16] K. Egashira, M. Vero, R. Staab, J. He, and M. Vechev, "Exploiting LLM quantization," in *Proc. NeurIPS*, 2024.
- [17] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018.
- [18] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017.